HedgeBERT: Hedge-Aware Sentiment Classification via CLS Feature Injection

Minh Tran
Wichita State University
K699E569@wichita.edu

An Nguyen
Wichita State University
R374G648@wichita.edu

Abstract

Hedging is the use of cautious or vague language that is often used to soften the force of a statement (Lakoff, 1973; Hyland, 1998). Hedging also causes problems for tasks such as sentiment analysis by causing models to fail when trying to capture the uncertainty of hedged sentences (Pang and Lee, 2008; Liu, 2012). This is because sentences like "I guess it is sort of decent" hold weakened sentiment, which makes it hard to correctly classify.

To solve this, we propose a novel two-stage framework for hedge detection. First, we create a hedge intensity scorer which grades a sentence on how hedged it is by looking for cue words, a method traditionally used in hedge detection corpora (Vincze et al., 2008; Farkas et al., 2010). We then match cue words based on certain categories with the weights of each category being learned through logistic regression. The second stage involves using the hedge intensity scores to fine-tune BERT (Devlin et al., 2019) to give us HedgeBERT, which is a model that is capable of capturing hedge sentiment to correctly classify hedged sentences.

1 Introduction

Uncertainty is a part of human language, and speakers often soften their claims when giving orders or reviewing products. This is achieved through linguistic devices such as epistemic verbs, modal auxiliaries, approximators, and diminishers (Prince et al., 1982; Lakoff, 1973). These categories are broadly classified under "hedging" which is defined as any linguistic means used to show a lack of commitment to the truth of a proposition or to avoid expressing that commitment categorically (Hyland, 1998). In the context of sentiment analysis, hedging obscures clear positive or negative signals. Correctly identifying sentiment is critical in domains like product reviews, where failing to account for hedged uncertainty can lead businesses to misinterpret public perception of their services (Pang and Lee, 2008; Liu, 2012). The main problem is that many models struggle to detect hedged language and correctly deal with it. Often causing them to incorrectly classify them.

In this work, we present a two-stage framework for hedge detection in review text. The first stage involves creating a hedge intensity scorer. We construct a cue inventory—drawing on established methods for identifying linguistic triggers in scientific and review corpora (Vincze et al., 2008; Szarvas, 2008)—to identify common hedging devices. We then assign weights to these devices based on their impact on sentiment, using a logistic regression model to learn these weights from data. This results in a scorer capable of quantifying the "hedge intensity" of any given sentence based on its constituent words.

In the second stage, we apply these intensity scores to reviews from the Amazon Electronics dataset and the SFU Review dataset (Konstantinova et al., 2012). These scores are then injected into the `[CLS]` token of a BERT model (Devlin et al., 2019) during fine-tuning, an architectural approach similar to recent work in sentiment-feature fusion (Fasulo et al., 2025b). We call this resulting model HedgeBERT. By explicitly training on intensity scores, the model can better detect patterns in uncertain language, allowing for more robust classification of hedged sentiment.

We benchmark HedgeBERT on a subset of the SFU Review dataset containing known sentiment labels. To measure performance, we compare HedgeBERT against a standard BERT baseline as well as other state-of-the-art transformer architectures, including ELECTRA (Clark et al., 2020) and BART (Lewis et al., 2020), to determine its efficacy in handling linguistic uncertainty compared to general-purpose models.

2 Related Works

2.1 Sentiment Analysis and Benchmarks

Sentiment analysis is a field that has a long history in natural language processing. In 2008, the foundational survey paper (Pang and Lee, 2008) provided techniques of opinion mining and early lexical classifiers and supervised machine learning approaches. Binary classification of sentiment analysis is among the most studied tasks in natural language processing with benchmark datasets such as SST-2 (Socher et al., 2013) and the IMDb large movie review corpus serving as the evaluation sets for works that followed after.

Despite how explored the field is, the benchmarks share a limitation: they are dominated by clearly polar examples and do not show performance on hedged or speculative language. SST-2 get its basis from film reviews parsed at the phrase level and filtered for clear sentiment signals. Neither dataset separate hedged and direct sentences or measures their accuracy from these subsets independently. It is possible that a classifier that evaluates well on the benchmarks but fail on hedged language that exists in real-world review text.

The closest related work in sentiment literature is SO-CAL (Taboada et al., 2011), a lexicon-based sentiment analysis system that addresses irrealis contexts, speculative or conditional framings that block sentiment propagation. SO-CAL applies rules that are hand-crafted to prevent clauses that have sentiment that are negated or speculative from contributing to the overall sentiment score. The work acknowledges hedging in language is a roadblock in sentiment analysis but does not have analysis on how hedges affect the evaluation, does not learn hedge weights from the data, or implement a feature for neural classifier on hedging. To our knowledge, no prior work directly measured hedged versus direct accuracy gap in neural sentiment classification or use a feature injection to gauge hedge intensity.

2.2 Hedging in Linguistics

The concept of hedging was introduced to linguistics by Lakoff (1973), who noticed that natural language does not operate on binary truth values. Words such as "sort of", "rather", and "technically" makes category membership a matter of degree instead of a binary fact. Lakoff's framing motivates our usage of a scalar instead of a binary hedged and not-hedged system. A sentence can be more or less hedged, not simply hedged or not hedged.

Prince et al. (1982) developed the first taxonomy of hedging in physician discourse, distinguishing approximators — words and phrases that modify the propositional content of a claim —- from shields, words and phrases that attributes claims to a speaker's subjective perspective. These two categories we adopt into our cue inventory keeping their taxonomy, but changing the name of shields to epistemic from another paper.

Hyland (1998) extended the study of hedging to to academic writing, establishing epistemic modality — the expression of a writer's degree of certainty or commitment to a proposition — as a feature of scientific discourse. The naming of shields as epistemic in our cue taxonomy is drawn from this framing. Observations from Hyland found that hedging is pervasive in expert writing which gave us motivation to extend the study the semantic weight in customer reviews.

Fraser (2010) further distinguishes propositional hedges, words and phrases that qualify certainty in the content of a claim, from relation hedges, words and phrases that manage the social relationship between speaker and listener. The distinction is partially overlapped with the four taxonomy from Raphalen et al. (2022) which corresponds to our approximators and epistemic categories.

These papers together establishes that hedging is a well-defined phenomenon with identifiable surface forms, predictable semantic effects, and a history of classification which provides the grounding of our cue inventory and category structure for our scorer.

2.3 Hedge Detection Corpora and NLP Systems

Hedge detection using computational resources emerged primarily in the biomedical natural language processing community motivated by the prevalence of speculative language in the scientific research literature. The Bioscope corpus (Vincze et al., 2008) is the foundational resource that contained 1,273 sentences from biomedical abstracts, full papers, and clinical reports that were annotated for speculation and negation at token and scope level. Importantly, the Bioscope annotates which words are hedge cues without grouping them into categories. Their classification of the hedge words were binary classification distinguishing from negation and speculation. Some of the hedge cues we use is derived from Bioscope while the taxonomy

was derived from hedging linguistic literature.

The CoNLL-2010 shared task (Farkas et al., 2010) established hedge detection as a competitive benchmark, using Bioscope as training data. The highest performing systems in the shared task used supervised learning over hand crafted keyword features, including Medlock and Briscoe (2007), who showed that key based weak supervision can effectively identify hedge cue and Szarvas (2008), who used keyword selection with logistic regression to classify hedge instances. Our weight learning procedure is a direct descendant of this approach: We used logistic regression over cue category counts to learn per-category hedge weights from annotated data.

A critical limitation of the biomedical hedge detection literature is for our purposes there is a mismatch in what hedging is used. The cues used in biomedical such as "suggest", "indicate", "appear" and "weather" are formal epistemic phrases that is commonly used in scientific literature. Consumers use informal epistemic phrases such as "I think", "I guess", "hard to say"; approximators such as "kind of" and "sort of"; and words that greatly reduces positive sentiments which we call diminishers such as "decent", "okay", and "not bad" which are missing from the Bioscope cue inventory. This led us to derive the cue inventory from the SFU Review Corpus (Konstantinova et al., 2012), which annotates speculation in consumer review over 400 reviews and eight product categories. To our knowledge, SFU is the only English-language customer review corpus with both speculation annotations and document-level sentiment labels, making it a suited as both a cue source and evaluation set for our tasks.

Recent hedge detection studies have expanded beyond the biomedical domain. Ghosal et al. (2022) introduces HedgePeer, a dataset of uncertainty annotation in scientific peer review. Raphalen et al. (2022) study hedging in spoken peer tutoring interactions, proposing four category taxonomy of propositional hedges, extenders, subjectivizer, and apologizers grounded in the literature of Prince et al. (1982) and Fraser (2010). The subjectivizer and propostional hedges categories overlap with the approximators and epistemic categories while the other categories share little to no similarities with our cue as they are lexicon related to in person speech. Neither of these papers are related to hedge detection for sentiment classification accuracy or propose a system for hedge-aware sentiment.

2.4 Feature-Augmented Transformers

Fine-tuning pre-trained transformers for downstream classification tasks use the representation of the `[CLS]` token as a sentence-level embedding, fed directly to a linear classification head Devlin et al. 2019. Numerous works have augmented this representation with external features to incorporate information that the transformer's self attention mechanism may not capture efficiently from text alone.

Wolf et al. (2020) document the general pattern of concatenating external feature vectors to transformer representation in multi-modal and multi-task settings involving dense embeddings using a second encoder. It also talks about using scalar feature injection in which a single continuous value derived from an independent analysis system is appended to the `[CLS]` vector before classification. This mechanism preserves the interpretability of the injected signal.

3 Datasets

3.1 SFU Review Corpus

The SFU Review Corpus (Konstantinova et al., 2012) is a collection of 400 consumer reviews with 16,211 sentences among those 400 reviews that span across 8 domains such as books, cars, computers, cookware, hotels, movies, music, and phones. Each of these reviews are annotated and labeled on whether their sentiment is positive or negative. The annotation is done through XML where each cue words are annotated like the following:

```
<cue ID="0" type="speculation"><W>possible</W
    ></cue>
```

The sentiment is found through the filename where if the filename starts with yes then the review holds positive sentiment and vice versa if it starts with no.

3.2 Amazon Reviews

The Amazon Review Dataset is large dataset that contains 571.54M reviews from interactions from 1996 to Sep. 2023. It has reviews from 34 categories like electronics, books, software, and video games. Each review has a text which is the review, a title, and a rating with a max score of 5. A rating of 5 represents a good review, while a rating of 1 represents a poor review.

3.3 Benchmark Construction

To evaluate our HedgeBERT we sample 256 sentences from the SFU review dataset. Out of the 256 sentences we make sure we have 128 positive, 128 negative, 128 hedged, and 128 direct sentences. So means that there are sentences that are both hedged and negative or direct and negative. Additionally to not bias a single domain we take 32 reviews from each of the 8 domains that were noted earlier and sentences are filtered to 6-60 words before sampling.

3.4 Training Set Construction

To fine-tune each of our models we mainly use the Amazon Reviews dataset where we take 6,800 positive reviews which correlate to 4* and 5* reviews as well as 6,800 negative reviews which correlate to 1* and 2* reviews. We did not include any 3* reviews as it is unsure whether they or positive or negative so we assumed them to be neutral. The number of reviews comes from us taking 400 reviews from each of the 34 categories with an even number of positive and negative reviews per category. We also combine the Amazon Reviews Dataset with a subset of the SFU Review Dataset that we didn't use for benchmarking and taking 1000 hedged sentences. By combining the two we get 14,600 reviews/sentences which we benchmark models on to check if they can correctly get the sentiment.

4 Hedge Intensity Scorer

The Hedge Intensity Scorer is a rule-based component that maps any sentence to a scalar $s \in [0.0, 1.0]$ representing how hedged that sentence is. It operates on the raw sentence string before tokenization, has no learnable parameters, and is fixed before any training begins. Its output serves as a single extra features injected into the HedgeBERT classification head. The scorer has two components: a weight learning procedure that derives per-category weights from corpus data, and a scoring formula that combines detected cue evidence into a single continuous score.

4.1 Cue Inventory

Cue words were derived by examining the set of tokens annotated as speculation cues in the SFU Review Corpus (Konstantinova et al., 2012). Because SFU annotates individual cue tokens without grouping them into named categories, the organization of the five categories is derived from the the theoretical and biomedical hedging literature.

Epistemic phrases. Twenty-two cues that attribute the sentiment claim to the speaker's subjective view: *I think, I believe, I feel, I guess, I suppose, I assume, I suspect, it seems, it appears, it looks like, in my opinion, in my view, to me, I would say, I'm not sure, I am not sure, not sure, hard to say, I wonder, I doubt, seems like, seems to*. The category name follows Hyland (1998) and corresponds to the shield category of Prince et al. (1982). The majority of these are multi-word expressions absent from the BioScope cue inventory, reflecting the informal first-person register of consumer reviews.

Modal Auxiliaries. Eight cues expressing epistemic possibilities: *might, could, would, may, should, can, ought to, need to*. Modals are a standard grammatical category in english (Vincze et al., 2008) and the most reliable hedge signal in the corpus. However we do notice that *should* and *can* are ambiguous as words as they can mean capability and recommendation while also meaning uncerntainty.

Approximators. Eighteen cues that weaken the precision of the modified expression without questioning its truth: *kind of, sort of, somewhat, rather, fairly, pretty much, quite, a bit, a little, slightly, more or less, roughly, approximately, around, almost, nearly, in a way, to some extent*. The category name is taken directly from Prince et al. (1982).

Diminishers. Fourteen cue words that bear sentiment. They express weak positive sentiment through understatement rather than an explicit hedge marker: *decent, acceptable, okay, fine, not bad, alright, all right, tolerable, adequate, average, mediocre, passable, satisfactory, sufficient*. Diminishers hedge by vocabulary choice rather than syntactic modification — selecting *decent* instead of *excellent* is an act of understatement that weakens the sentiment signal without any additional cue token. The category was motivated by the frequency of such understatement expressions in the SFU consumer review data.

Conditionals. Seven cues that make the expressed sentiment reliant on a condition being met: *if, unless, provided that, as long as, assuming that, in case, supposing*. Conditional cues are present in both the Bioscope and SFU and function by restrict-

ing how applicable the sentiment is to a specific hypothetical context.

The full inventory totals 69 cue words and phrases across five categories. Table 1 shows representative examples from each category.

Category	Example cues
Epistemic	*I think, I guess, hard to say*
Modal	*might, could, may, would*
Approximator	*kind of, sort of, a bit*
Diminisher	*decent, okay, not bad*
Conditional	*if, unless, provided that*

Table 1: Representative cues from each category in the hedge cue inventory.

4.2 Weight Learning

Rather than setting cue category weight by hand, we learn them from the SFU corpus using logistic regression, following the supervised keyword weighting approach of Medlock and Briscoe (2007) The intuition is that a category's weight should reflect how strongly its presence predicts hedging observed in the human annotated data.

Feature construction. For each 15,952 non-benchmark SFU sentences, we extract a five-dimensional integer feature vector $\mathbf{x} = (x_1, \ldots, x_5)$ where each element x_k is the count of distinct cue words from category k detected in that sentence, subject to the scope constraint found under the scope handling section. The binary target label $y \in \{0, 1\}$ is the `is_hedged` flag derived from SFU's human annotations. The 256 benchmark sentences are excluded from this procedure entirely to prevent data leakage.

Class Imbalance. The SFU corpus contains approximately 3,774 hedged sentences and 12,178 direct sentences, a ratio of roughly 1:3. We address this using `class_weight="balanced"` in scikit-learn's `LogisticRegression`, which upweights the loss contribution of each hedged training example by a factor of about 3.23, preventing the classifier from defaulting to the majority class.

Model and optimization. We use binary logistic regression with L2 Regularization ($C = 1.0$), solved by L-BFGS with `max_iter=1000` to ensure convergence. The five-dimension feature space is suited for logistic regression as the model is unlikely to overfit with 15,952 training examples and five features. The learned coefficients are directly interpretable as per-category predictive weights.

Coefficient extraction and normalization. Raw coefficients β_k are extracted from the fitted classifier as `clf.coef_[0]`.Two post-processing steps are applied. First, negative coefficient is clipped to zero: a hedge cue category should not reduce the hedge score, so any negative coefficient is treated as noise rather than an anti-hedge signal. Second, the clipped coefficients are normalized to $[0.1, 0.9]$ using min-max scaling. The floor of 0.1 ensures every surviving category contributes at least minimally when a cue is detected; the ceiling of 0.9 prevents any single category from saturating the score and suppressing the contribution of additional cues.

Learned Weights. The resulting weights are shown in Table 2. Modal auxiliaries received the highest weight (.9000), consistent with constant usage in as hedge markers in consumer reviews. Conditional cues are the second strongest weights (.7686), reflecting that conditional clauses restrict sentiment applicability. Epistemic phrases receive a moderate weight (0.5411), indicating that first-person attribution has a good chance to be a hedge signal. Approximators (0.1181) and diminishers (0.1000) receive weak weights, which is informative: both categories appear at similar rates in hedged and direct sentences, making them unreliable predictors of the `is_hedged` label despite being genuine hedge markers at the semantic level. The approximator and diminisher weights hit or approach the normalization floor, reflecting the ambiguity of words like *kind of* and *decent* across different review contexts.

Category	Weight
Modal	0.9000
Conditional	0.7686
Epistemic	0.5411
Approximator	0.1181
Diminisher	0.1000

Table 2: Learned per-category hedge weights after clipping and min-max normalisation to $[0.1, 0.9]$. Weights reflect how strongly each category predicts `is_hedged` in the SFU corpus.

4.3 Scoring Formula

Given a sentence with N detected cue instances, each with weight w_i, the hedge intensity score is computed as:

$$s = 1 - \prod_{i=1}^{N} (1 - w_i) \tag{1}$$

This is the Noisy-OR model (Pearl, 1988), which computes the probability that at least one of N independent causes produces an effect. Each cue i is treated as an independent cause with probability w_i of being sufficient evidence for hedging. The formula gives the probability that at least one cue is sufficient — that is, that the sentence is hedged by at least one of its detected cues.

Boundary cases. When no cues are detected, the empty product equals one by convention, giving $s = 0.0$. When a single cue of weight w is detected, $s = 1 - (1 - w) = w$ exactly, so the single-cue score equals the learned category weight directly.

Diminishing returns. Each additional cue contributes less than the one before it. The marginal gain of adding cue k to an existing set of detected cues is:

$$\Delta_k = w_k \cdot \prod_{i<k}(1 - w_i) \tag{2}$$

As more cues are detected the remaining product shrinks, and the marginal gain of any additional cue shrinks proportionally. This prevents the score from being inflated by many weak cues: three approximators at $w = 0.1181$ each produce $s = 1 - (0.8819)^3 = 0.313$, which is substantially lower than a single modal at $s = 0.900$. This behavior reflects linguistic intuition — *I think this might be sort of decent* is not more hedged than *this might be decent*; the additional cues confirm rather than increase the hedge.

Independence assumption. The Noisy-OR model assumes that cues fire independently. This assumption is violated in practice: modal auxiliaries and epistemic phrases tend to co-occur in genuinely uncertain sentences more than chance would predict, meaning the formula may slightly underestimate hedge intensity for multi-cue sentences. This is an accepted approximation in exchange for the model's interpretability and computational simplicity.

Worked example. Consider the sentence *"I think the product worked decent enough but is hard to say."* The scorer detects three cues: *I think* (epistemic, $w = 0.5411$), *hard to say* (epistemic, $w = 0.5411$), and *decent* (diminisher, $w = 0.1000$). Applying Equation 1:

$$\begin{aligned} s &= 1 - (1 - 0.5411)(1 - 0.5411)(1 - 0.1000) \\ &= 1 - (0.4589)(0.4589)(0.9000) \\ &= 1 - 0.1895 \\ &= 0.8105 \end{aligned}$$

The sentence scores 0.8105, correctly reflecting heavy hedging from two epistemic cues and one diminisher.

4.4 Scope Handling

A naive scorer that credits cues from anywhere in the sentence may over-estimate hedge intensity for compound sentences where hedging is confined to a subordinate clause. For example, in *"The camera is excellent but I think the battery might die,"* the cues *I think* and *might* belong to the second clause and should not inflate the hedge score of the positive main clause.

To address this, we apply a conjunction-boundary scope simplification. Before cue detection, the sentence is split at nine conjunction boundary tokens: *but, although, though, however, yet, while, whereas, nevertheless, despite*. A cue is credited only if its regex match falls within the same clause in which it is detected — that is, cues are not credited across clause boundaries.

Scope handling is applied identically during weight learning and during scoring, ensuring that the logistic regression weights are trained on the same cue representation used at inference time.

We acknowledge that conjunction-boundary splitting is a partial solution. It handles the most common compound sentence pattern but does not resolve embedded clauses, relative clauses, or more complex syntactic structures. Full clause-level scope resolution would be direction for future work. For the majority of consumer review sentences, which are syntactically simple single-clause constructions, scope splitting has little to no effect and the scorer operates on the whole sentence by default.

4.5 Pre-computation and Integration

The scorer is implemented as a standalone Python class, `HedgeScorer`, which loads `weights.json` at initialisation and compiles whole-word regex patterns for all 69 cues. Patterns are sorted by cue length in descending order so that multi-word expressions (*more or less*, *I'm not sure*) are matched before their sub-phrase components (*more*, *sure*), preventing sub-phrase collisions.

Each cue string is counted at most once per sentence regardless of how many times it appears, us-

ing a `seen_cues` set during detection. This prevents repetition — such as *I think... I think* — from artificially inflating the score.

Before HedgeBERT training begins, hedge scores are pre-computed for all 2,000 sentences in the combined training set and stored alongside the training data. During the training loop, scores are retrieved by index rather than recomputed on each batch, avoiding redundant regex operations across epochs. At evaluation time, scores for the 256 benchmark sentences are computed on the fly using a module-level cached scorer instance, ensuring the scorer is exercised in its real inference setting.

5 Models

5.1 BERT

BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019) is an transformer based (Vaswani et al., 2017) encoder model. It has two objectives which are predicting masked tokens from other surrounding context words and next sentence prediction. BERT was chosen as our base model for HedgeBERT as well as the benchmark to compare HedgeBERT against.

5.2 HedgeBERT

HedgeBERT is our BERT based model and our primary contribution. Rather than just fine-tuning it on hedged positive or negative sentences we are fine-tuning it on a hedge intensity score which is being injected into the CLS token. The score is derived as seen in Section 4 and as a result of the score the model is able to be more sensitive to the presence and degree of hedging.

5.3 ELECTRA

ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) (Clark et al., 2020) is a transformer encoder like BERT but was trained to identify was words in a sequence were replaced by a small generator. This makes training ELECTRA much more efficient so we decided to include ELECTRA to see if a more efficient pretraining offers better advantages on hedge detection.

5.4 Small BERT

Small BERT was included due to us wanting to see the trade-off between model capacity vs performance on hedge detection. We also wanted to see if a small HedgeBERT would be able to compete against the normal BERT when it comes to hedge detection in review texts. Comparing against HedgeBERT will also allow us to assess how much of a performance gap there is between scale of models.

5.5 BART

BART (bidirectional and Auto-Regressive Transformers) (Lewis et al., 2020) is a sequence-to-sequence model which combines an bidirectional encoder with a auto regressive decoder. BART was mainly designed for generation tasks like summarization, but due to its encoder part its able to capture some sentiment which is why we included it.

5.6 GPT-3

GPT (Brown et al., 2020) is a decoder-only transformer (Vaswani et al., 2017) and is much bigger with 175B parameters. This is much bigger than our other models and has a different architecture than the other models. We included GPT-3 in our tests due to its size and architecture as we wanted to see if its emergent capabilities noted in its paper could extend to hedged language.

5.7 LLaMA2

LLaMA2 (Touvron et al., 2023) is a family of models, for our purposes we specifically used the 7 billion version. It is a decoder only transformer like GPT-3, but differs in some aspects as LLaMA2 uses rotary positional embeddings instead of the absolute positional embeddings that GPT-3 uses. We included LLaMA2 7b as a open-weight decoder model as GPT-3 isn't as publicly accessible.

6 Experimental Setup

6.1 Training Procedure

BERT, HedgeBERT, ELECTRA, small BERT, small HedgeBERT, and BART were all fine-tuned on the training data with the same procedure. First the training data was shuffled and split into training and validation sets. Then all sentences are tokenized with each model's tokenizer then fine-tuned for 5 epochs with binary cross-entropy loss. Additionally for HedgeBERT and smalle HedgeBERT the hedge-intensity scorer is used to create a hedge score for each sentence which is added to the `[CLS]` token. This procedure is done for the 5

models listed and for GPT-3.5-turbo and LLaMA 2 7B, they are just evaluated in zero-shot prompting.

6.2 Fine-tuned Models

All fine-tuned models share an identical training configuration, summarised in Table 3.

Hyperparameter	Value
Epochs	5
Learning rate	2e-5
Optimiser	AdamW (Loshchilov and Hutter, 2019)
Loss	CrossEntropyLoss
Max sequence length	128 tokens
Mixed precision	fp16 (GradScaler + autocast)
Checkpoint selection	Best validation accuracy

Table 3: Shared hyperparameter configuration for all fine-tuned models.

Vanilla BERT baseline. `bert-base-uncased` (Devlin et al., 2019) with a standard `Linear(768→2)` classification head applied to the `[CLS]` token, preceded by `Dropout(0.1)`. No hedge information is injected. This model establishes the baseline hedged versus direct accuracy gap without any intervention and is the primary comparison point for HedgeBERT.

HedgeBERT. The same `bert-base-uncased` backbone, extended by concatenating the hedge intensity scalar $s \in [0.0, 1.0]$ produced by the scorer to the 768-dimensional `[CLS]` vector. The `[CLS]` vector $\mathbf{c}$ (768 dimensions) is concatenated with the hedge scalar s to produce a 769-dimensional input $\mathbf{h} = [\mathbf{c} \,||\, s]$ to the classification head. The classification head is `Dropout(0.1)` followed by `Linear(769→2)`, initialised with Xavier uniform weights (Glorot and Bengio, 2010). Hedge scores are pre-computed for all training sentences and retrieved by index during training; benchmark scores are computed on the fly at evaluation time.

Ablation: random scalar. Architecture identical to HedgeBERT. The hedge score is replaced at both training and evaluation time with a draw from $\mathrm{Uniform}(0, 1)$ at a fixed seed. The model never receives a meaningful hedge signal. This ablation isolates whether the real scorer contributes signal beyond a random noise dimension appended to the `[CLS]` vector.

ELECTRA Small. `google/electra-small-discriminator` (Clark et al., 2020) via HuggingFace `AutoModelForSequenceClassification`. No hedge injection. Included as a smaller pre-trained model baseline with a discriminative rather than masked pre-training objective.

Small BERT. `google/bert_uncased_L-4_H-256_A-4` via `AutoModelForSequenceClassification`. A four-layer, 256-hidden-dimension BERT variant (Devlin et al., 2019). No hedge injection. Included as a lightweight BERT baseline.

BART Base. `facebook/bart-base` (Lewis et al., 2020) via `AutoModelForSequenceClassification`. No hedge injection. BART's denoising sequence-to-sequence pre-training objective differs substantially from BERT's masked language modelling, providing a baseline with a different inductive bias.

6.3 Zero-Shot LLM Baselines

two large language models are evaluated zero-shot to test whether the hedged versus direct accuracy gap persists at scale without fine-tuning.

GPT-3.5-turbo GPT-3 is queried via the OpenAI Chat Completions API with `temperature=0` for deterministic output and `max_tokens=5` to constrain the response to a single label. The prompt is:

> *System:* You are a sentiment classifier. Classify the sentiment of the given sentence as either positive or negative. Respond with exactly one word: positive or negative. No explanation.
>
> *User:* Sentence: "{sentence}"
> Sentiment (positive or negative):

Responses that begin with neither *positive* nor *negative* (case-insensitive) are counted as skipped.

LLaMA-2-7b-chat. Evaluated via HuggingFace using 4-bit NF4 quantization (Touvron et al., 2023) with bfloat16 compute. The LLaMA 2 Chat `[INST]` prompt template is used with greedy decoding (`do_sample=False`, `max_new_tokens=10`). The parser reads the first 50 characters of the generated response and searches for *positive* or *negative*.

6.4 Evaluation Metrics

All models are evaluated on the 256-sentence benchmark using three primary quantities:

1. **Hedged subset accuracy** — accuracy on the 128 sentences containing at least one SFU speculation annotation.

2. **Direct subset accuracy** — accuracy on the 128 sentences with no speculation annotations.

3. **Accuracy gap** — direct accuracy minus hedged accuracy. A positive gap indicates worse performance on hedged sentences, consistent with the project hypothesis. A negative gap indicates the reverse.

Macro F1 is reported as a secondary metric for each subset to confirm that high accuracy does not conceal degenerate prediction behaviour. Because the benchmark is perfectly balanced (64 positive and 64 negative per subset), accuracy and macro F1 are expected to be close, and accuracy is the primary reported figure. Overall benchmark accuracy is reported but is secondary to the gap between hedged and direct.

7 Results

Table 5 presents benchmark performance across seven models all are evaluated on the benchmark noted in section 3.3 as well as the hedged and direct subsets of the benchmark independently. The last column GAP(D-H) is calculated as the direct accuracy - hedged accuracy

Overall Performance Out of all the models that were evaluated, GPT-3.5-turbo came out with the highest accuracy and Macro F1 scores which implies that the larger scale does play a role in achieving better results. Moving onto the fine-tuned models which are much smaller, BART base came out with the highest overall accuracy which is only a margin better than HedgeBERT and Bert base. This is consistent as GPT-3.5-turbo was the best overall and also the biggest while BART base is the biggest among the fine-tuned models which shows that a bigger model tends to do better overall.

Hedged Performance Among all the models, BART and HedgeBERT tie for highest hedged accuracy. However, comparing HedgeBERT to BERT base, there is a clear difference in hedged accuracy with HedgeBERT being 0.039 above BERT base. Similar results are also seen which comparing Small BERT and Small HedgeBERT with Small HedgeBERT doing better than most of the models. This shows that injecting the hedged scalar into the `[CLS]` token does produce results that align with the goal.

Direct vs Hedge Gap The last column gap shows the difference between direct and hedged accuracies. The first thing to note is that the gap for all models is positive which means that each model did better on direct reviews than hedged reviews. This shows that the problem of models being able to detect and correctly classify hedged sentences is a real issue. Additionally, HedgeBERT had the lowest gap among all models which suggests that it is able to see hedged and direct sentences the roughly the same.

Summary of Results All together, the results show two main things. First, the problem of hedged sentences being harder to classify and detect is a real issue which is shown by the gap. Second, the injection of a hedge scalar does help with hedge detecting and produces a measurable result.

7.1 Ablation Study

To determine if its actually from the hedge intensity scorer, we run an ablation study by instead of using the hedge intensity scorer we use a random scalar that fits in the same range of the intensity scorer. All training is done the same way except for the random scalar.

Model	Hedged Acc.	Direct Acc.	Gap (D−H)
HedgeBERT	**0.625**	0.641	**+0.016**
Random scalar	0.617	0.648	+0.031
BERT base	0.586	**0.680**	+0.094

Table 4: Ablation study results with same setup as original setup.

Results for the ablation show that HedgeBERT still has the highest accuracy but the random scalar does show an increased hedged accuracy. Both HedgeBERT and the random scalar outperform the BERT base which suggests that just having the scalar there does have some affect. Additionally HedgeBERT is still .8 percent above the random scalar on hedged accuracy which implies that the hedged scalar does bring some meaningful information.

8 Limitations

We acknowledge that there are some limitations in our method. First, the hedge scorer on detecting words/phrases from a hard coded list which means we are looking for a fixed amount of words. As a result some words or phrases could be getting missed and reduces the quality of the hedge

Model	Setting	Overall Acc.	Macro F1	Hedged Acc.	Direct Acc.	Gap (D−H)
Zero-shot (no fine-tuning)						
GPT-3.5-turbo	zero-shot	**0.652**	**0.652**	0.605	**0.698**	+0.094
LLaMA 2 7B	zero-shot	0.602	0.601	0.551	0.654	+0.102
Fine-tuned						
BART base	fine-tuned	**0.641**	**0.641**	**0.625**	**0.656**	**+0.031**
HedgeBERT	fine-tuned	0.633	0.632	**0.625**	0.641	**+0.016**
BERT base	fine-tuned	0.633	0.632	0.586	0.680	+0.094
ELECTRA small	fine-tuned	0.617	0.608	0.555	0.680	+0.125
Small BERT	fine-tuned	0.598	0.598	0.547	0.648	+0.102
Small HedgeBERT	fine-tuned	0.602	0.602	0.578	0.625	+0.047

Table 5: Benchmark results across all models. Overall accuracy and macro F1 are computed on the full 256-sentence benchmark. Hedged and direct accuracy are computed on the 128-sentence subsets respectively. Gap (D−H) denotes the accuracy difference between direct and hedged subsets; Best result in each group is shown in **bold**.

scorer by making it less meaningful. Second, the benchmark is only 256 sentences due to the size of the SFU Review dataset being small which limits the confidence in the results. Lastly, the ablation shows the random scalar performing nearly as well as HedgeBERT on hedged accuracy which limits the claims of our work. However we believe this is due to the quality of our hedge scorer being not the best it could be.

9 Conclusions

In our work, we presented a two-stage framework for hedge-aware sentiment classification by creating a hedge intensity scorer to give sentences a hedge intensity score based on five categories. We then use the hedge intensity score to help fine-tune BERT producing HedgeBERT. Our results show that the hedging problem does exist where models struggle when it comes to correctly guessing sentiment on hedged sentences. Furthermore, our results show that injecting a hedge intensity score into the `[CLS]` token does help in detecting hedged language. The most significant challenge was the absence of a large high quality dataset that confidently annotates words that indicate hedging, the sentiment of the sentence, and whether the sentence is hedged or not. Several future directions can be taken from this. The first being to replace the hedge intensity scorer with a model that will more accurately give the hedge intensity of a sentence. Second expanding the benchmark will allow for a stronger conclusion on the performance and possibly make a difference on the ablation study giving a more conclusive result.

References

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *International Conference on Learning Representations*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Richárd Farkas, Veronika Vincze, György Móra, János Csirik, and György Szarvas. 2010. The CoNLL-2010 shared task: Learning to detect hedges and their scope in natural language text. In *Proceedings of the Fourteenth Conference on Computational Natural Language Learning — Shared Task*, pages 1–12, Uppsala, Sweden. Association for Computational Linguistics.

Alessandro Fasulo and 1 others. 2025a. Checkthat! at CLEF 2025: Subjectivity and sentiment detection. In *Proceedings of the 16th International Conference of the CLEF Association*.

Matteo Fasulo, Luca Babboni, and Luca Tedeschini. 2025b. AI Wizards at CheckThat! 2025: Enhancing transformer-based embeddings with sentiment for

subjectivity detection in news articles. In *Working Notes of CLEF 2025 - Conference and Labs of the Evaluation Forum*, volume 4038 of *CEUR Workshop Proceedings*, pages 912–925. CEUR-WS.org.

Bruce Fraser. 2010. Hedging in political discourse. In Urszula Okulska and Piotr Cap, editors, *Perspectives in Politics and Discourse*, pages 201–213. John Benjamins, Amsterdam.

Devamanyu Ghosal, Wenjie Shen, Navonil Majumder, Rada Mihalcea, and Soujanya Poria. 2022. HedgePeer: A dataset for uncertainty detection in scientific peer reviews. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2113–2124, Dublin, Ireland. Association for Computational Linguistics.

Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256. JMLR Workshop and Conference Proceedings.

Ken Hyland. 1998. *Hedging in Scientific Research Articles*. John Benjamins Publishing, Amsterdam.

Natalia Konstantinova, Sheila C. M. de Sousa, Noa P. Cruz, Manuel J. Maña, Maite Taboada, and Ruslan Mitkov. 2012. A review corpus annotated for negation, speculation and their scope. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, pages 3298–3300, Istanbul, Turkey. European Language Resources Association (ELRA).

George Lakoff. 1973. Hedges: A study in meaning criteria and the logic of fuzzy concepts. *Journal of Philosophical Logic*, 2(4):458–508.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.

Bing Liu. 2012. *Sentiment Analysis and Opinion Mining*. Morgan & Claypool Publishers.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA. Association for Computational Linguistics.

Ben Medlock and Ted Briscoe. 2007. Weakly supervised learning for hedge classification in scientific literature. In *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics*, pages 992–999, Prague, Czech Republic. Association for Computational Linguistics.

Roser Morante and Walter Daelemans. 2012. ConanDoyle-neg: Annotation of negation in Conan Doyle stories. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, pages 1563–1568, Istanbul, Turkey. European Language Resources Association (ELRA).

Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 188–197, Hong Kong, China. Association for Computational Linguistics.

Bo Pang and Lillian Lee. 2008. Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1–2):1–135.

Judea Pearl. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Francisco, CA.

Ellen F. Prince, Joel Frader, and Charles Bosk. 1982. On hedging in physician-physician discourse. In Robert J. Di Pietro, editor, *Linguistics and the Professions*, pages 83–97. Ablex, Norwood, NJ.

Théo Raphalen, Chloé Clavel, and Daniel Luzzati. 2022. "I'm not sure this is right": Detecting hedge phrases in peer tutoring interactions. In *Proceedings of the 2022 ACL Workshop on Computational Approaches to Subjectivity, Sentiment & Social Media Analysis*, pages 42–52. Association for Computational Linguistics.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, Seattle, Washington, USA. Association for Computational Linguistics.

György Szarvas. 2008. Hedge classification in biomedical texts with a weakly supervised selection of keywords. In *Proceedings of the 46th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 281–289, Columbus, Ohio. Association for Computational Linguistics.

Maite Taboada, Julian Brooke, Milan Tofiloski, Kimberly Voll, and Manfred Stede. 2011. Lexicon-based methods for sentiment analysis. *Computational Linguistics*, 37(2):267–307.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008.

Veronika Vincze, György Szarvas, Richárd Farkas, György Móra, and János Csirik. 2008. The BioScope corpus: Biomedical texts annotated for uncertainty, negation and their scopes. *BMC Bioinformatics*, 9(Suppl 11):S9.

Jason Wei and Kai Zou. 2019. EDA: Easy data augmentation techniques for boosting performance on text classification tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6382–6388.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.